Take-Home Assignment: Chat-Based Data Analysis on Multiple CSVs

Objective

Develop a Proof of Concept (PoC) that enables users to chat with multiple CSV files simultaneously for data analysis.
Users should be able to ask questions about the dataset and receive responses in two possible formats:

1. Natural Language Response – A text-based explanation of insights.
2. Charts/Graphs – Visual representations of the data when applicable.

The system can intelligently mix these response types based on the query.

Scope & Requirements

Multi-CSV Support

- The system must allow users to upload multiple CSV files at once.
- Queries should be able to analyze data across different files (e.g., "Compare sales across regions from two different CSVs").

Conversational Interface

- Users should be able to ask natural language questions like:
  - "What is the total revenue for the past year?"
  - "Show me a sales trend over the last 6 months."
- The AI should process the question, fetch relevant data, and return a relevant response (text or visualization).

Response Generation

- Natural language responses for explanations, summaries, and insights.
- Charts/graphs when visualization is more meaningful (e.g., time trends, category comparisons).

Data Handling

- Perform data cleaning before analysis (handle missing values, data types, outliers, etc.).

- [Optional] Ensure the system can efficiently handle large CSV files.

Frontend & Backend

- The system must have both a frontend and a backend.
- Frontend:
    - Can be built using Next.js or React.
    - Should allow users to upload CSVs, ask questions, and view responses (text and visualizations).
- Backend:
    - Must be implemented as an API (FastAPI, Flask, Django, etc.) that the Next.js/React frontend communicates with over HTTP.
    - Must handle query processing, data analysis, and response generation.

Implementation Constraints

- The backend must be built using Python. The frontend must be built using Next.js or React.
- Candidates may use any framework/library for implementation. Backend (Python): Pandas, FastAPI, OpenAI API, FAISS, ChromaDB, Matplotlib, Plotly. Frontend (Next.js/React): any UI and charting libraries (e.g., Recharts, Chart.js, Plotly.js).
- Production-ready code: Proper project structure, modularity, and dependency management are required.

Evaluation Strategy

- The candidate does not need to write evaluation code but must describe how they would evaluate the system in a separate section.
- Evaluation aspects to be covered:
    - How to validate the accuracy of AI-generated insights.
    - How to test system performance on large datasets.
    - Edge cases and robustness testing.

Project Deliverables

1. GitHub Repository (Candidate must push the code to GitHub and share the link).
2. Production-Ready Code (Modular structure, well-documented).
3. README File explaining:
    a. The problem statement.
    b. How the system works.
    c. How to set up and run the project.
    d. Libraries used and why.
    e. How evaluation could be performed.

Bonus Points

- Multi-turn conversation support (e.g., follow-up questions like "Now break it down by category").
- Dockerfile for the frontend and backend